

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №2
по дисциплине «Построение и анализ алгоритмов»
Тема: «Расстояние Левенштейна.»
Вариант 4а

Студент гр. 4343

Стариков Н.С.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2026

Задание

1. Над строкой ε (будем считать строкой непрерывную последовательность из латинских букв) заданы следующие операции:

$\text{replace}(\varepsilon, a, b)$ – заменить символ a на символ b .

$\text{insert}(\varepsilon, a)$ – вставить в строку символ a (на любую позицию).

$\text{delete}(\varepsilon, b)$ – удалить из строки символ b .

Каждая операция может иметь некоторую цену выполнения (положительное число).

Даны две строки A и B , а также три числа, отвечающие за цену каждой операции. Определите минимальную стоимость операций, которые необходимы для превращения строки A в строку B .

Входные данные: первая строка – три числа: цена операции replace , цена операции insert , цена операции delete ; вторая строка – A ; третья строка – B .

Выходные данные: одно число – минимальная стоимость операций.

2. Над строкой ε (будем считать строкой непрерывную последовательность из латинских букв) заданы следующие операции:

$\text{replace}(\varepsilon, a, b)$ – заменить символ a на символ b .

$\text{insert}(\varepsilon, a)$ – вставить в строку символ a (на любую позицию).

$\text{delete}(\varepsilon, b)$ – удалить из строки символ b .

Каждая операция может иметь некоторую цену выполнения (положительное число).

Даны две строки A и B , а также три числа, отвечающие за цену каждой операции. Определите последовательность операций (редакционное предписание) с минимальной стоимостью, которые необходимы для превращения строки A в строку B .

Входные данные: первая строка – три числа: цена операции replace , цена операции insert , цена операции delete ; вторая строка – A ; третья строка – B .

Выходные данные: первая строка – последовательность операций (M – совпадение, ничего делать не надо; R – заменить символ на другой; I – вставить

символ на текущую позицию; D – удалить символ из строки); вторая строка – исходная строка A; третья строка – исходная строка B.

3. Расстоянием Левенштейна назовём минимальное количество операций вставки одного символа, удаления одного символа и замены одного символа на другой, необходимых для превращения одной строки в другую. Разработайте программу, осуществляющую поиск расстояния Левенштейна между двумя строками.

Пример:

Для строк `pedestal` и `stien` расстояние Левенштейна равно 7:

Сначала нужно совершить четыре операции удаления символа: `pedestal` -> `stal`.

Затем необходимо заменить два последних символа: `stal` -> `stie`.

Потом нужно добавить символ в конец строки: `stie` -> `stien`.

Параметры входных данных:

Первая строка входных данных содержит строку из строчных латинских букв. (S , $1 \leq |S| \leq 2550$).

Вторая строка входных данных содержит строку из строчных латинских букв. (T , $1 \leq |T| \leq 2550$).

Параметры выходных данных:

Одно число L , равное расстоянию Левенштейна между строками S и T .

4. Базовая часть ЛР № 3 остаётся неизменной. Индивидуализация:

Добавляется 4-я операция со своей стоимостью: замена одного символа на два символа.

Описание алгоритмов и оценка их сложности во времени и памяти

Во всех четырёх задачах используется метод динамического программирования (алгоритм Вагнера–Фишера) для вычисления минимальной стоимости преобразования одной строки в другую. Строки имеют длины $n=|A|$ и $m=|B|$. Основная идея – заполнение таблицы $dp[i][j]$, где i – количество обработанных символов из первой строки, j – из второй. Рекуррентное соотношение базируется на трёх стандартных операциях (удаление, вставка, замена), а в варианте 4а добавляется четвёртая операция (замена одного символа на два).

Во всех реализациях принято допущение, что стоимости операций удовлетворяют неравенству треугольника: замена двух последовательных операций одной не увеличивает общую стоимость. Это свойство гарантирует корректность динамического программирования для взвешенных расстояний. Также действует упрощающее условие: операции применяются только к исходным символам (строка преобразуется слева направо), что особенно важно для операции $1 \rightarrow 2$.

4.1. Взвешенное редакционное расстояние (только стоимость)

В задаче 4.1 вычисляется взвешенное редакционное расстояние — минимальная стоимость преобразования строки A в строку B с учётом заданных стоимостей операций удаления, вставки и замены. Для этого строится матрица dp размером $(n+1)$ на $(m+1)$, где n — длина A , m — длина B . Базовые случаи заполняются так: для любого i от 1 до n значение $dp[i][0]$ равно i , умноженному на стоимость удаления, что соответствует удалению всех символов из A . Для любого j от 1 до m значение $dp[0][j]$ равно j , умноженному на стоимость вставки, что соответствует вставке всех символов из B . Затем для каждой ячейки (i,j) с i от 1 до n и j от 1 до m вычисляется минимальное значение среди трёх вариантов: первый вариант — удаление символа $A[i-1]$, то есть $dp[i-1][j]$ плюс стоимость удаления; второй вариант — вставка символа $B[j-1]$, то есть $dp[i][j-1]$ плюс стоимость вставки; третий вариант — замена или

совпадение: если символы $A[i-1]$ и $B[j-1]$ равны, то добавляется 0, иначе добавляется стоимость замены, и всё это прибавляется к $dp[i-1][j-1]$. После заполнения всей таблицы ответом служит значение $dp[n][m]$.

Оценка сложности: по времени алгоритм работает за $O(n*m)$, поскольку для каждой из $n*m$ ячеек выполняется константное число операций.

По памяти требуется $O(n*m)$ для хранения всей матрицы dp .

4.2. Редакционное предписание (восстановление последовательности операций)

В задаче 4.2 для восстановления последовательности операций помимо матрицы dp заполняется вспомогательная матрица $parent[i][j]$, в которой для каждой ячейки запоминается операция, приведшая к минимальному значению: D для удаления, I для вставки, M для совпадения (без замены) и R для замены. Чтобы разрешить неоднозначности, когда несколько операций дают одинаковую стоимость, задаётся фиксированный приоритет: сначала I, затем D, затем R, причём внутри R предпочтение отдаётся M. После заполнения матриц выполняется обратный ход от ячейки (n,m) до $(0,0)$: в зависимости от значения $parent[i][j]$ перемещаются указатели и операция записывается в стек. Затем стек разворачивается, и итоговая последовательность выводится в виде строки из символов I, M, D, R. Отдельно выводятся исходные строки A и B.

Оценка сложности: по времени затраты составляют $O(n*m)$ на заполнение dp и $parent$ плюс $O(n+m)$ на обратный проход, то есть асимптотически доминирует $O(n*m)$.

По памяти требуется $O(n*m)$ для хранения двух матриц dp и $parent$; каждая ячейка $parent$ хранит один символ, поэтому общий объём памяти примерно вдвое больше, чем в задаче 4.1. Дополнительно используется стек для операций размером $O(n+m)$.

4.3. Расстояние Левенштейна (единичные стоимости)

В задаче 4.3 вычисляется расстояние Левенштейна — частный случай задачи 4.1, в котором стоимости замены, вставки и удаления равны единице. Матрица dp заполняется по тем же формулам, но без чтения стоимостей из входных данных — они подразумеваются единичными. Базовые случаи: $dp[i][0]$ равно i (удалить i символов), $dp[0][j]$ равно j (вставить j символов). Рекуррентное соотношение для ячейки (i, j) выбирает минимум из трёх значений: удаление символа $A[i-1]$ ($dp[i-1][j]$ плюс 1), вставка символа $B[j-1]$ ($dp[i][j-1]$ плюс 1) и замена или совпадение: если символы $A[i-1]$ и $B[j-1]$ равны, то берётся $dp[i-1][j-1]$, иначе $dp[i-1][j-1]$ плюс 1.

Оценка сложности: по времени алгоритм работает за $O(n*m)$, аналогично предыдущим задачам.

По памяти требуется $O(n*m)$ для хранения матрицы dp .

4.4. Расширенный вариант (добавлена операция замены одного символа на два)

В задаче 4.4 к трём стандартным операциям добавляется четвёртая: замена одного символа $A[i-1]$ на два символа $B[j-2]$ и $B[j-1]$ с заданной стоимостью $cost_replace1to2$. Эта операция допустима только при $j \geq 2$. Рекуррентное соотношение для заполнения матрицы dp дополняется соответствующим слагаемым: значение $dp[i][j]$ выбирается как минимум из удаления ($dp[i-1][j] + cost_delete$), вставки ($dp[i][j-1] + cost_insert$), замены одного символа на один ($dp[i-1][j-1]$ плюс 0 при совпадении символов или $cost_replace$ при несовпадении) и замены одного символа на два ($dp[i-1][j-2] + cost_replace1to2$) для j от 2. Для наглядности работы алгоритм дополнительно выводит промежуточные таблицы dp и $parent$, легенду с пояснением обозначений, а также последовательность операции. В матрице $parent$ для операции замены $1 \rightarrow 2$ используется символ 2.

Оценка сложности: по времени алгоритм остаётся $O(n*m)$, поскольку добавление четвёртого варианта проверяется за константное время в каждой ячейке.

По памяти требуется $O(n*m)$ для хранения матриц `dp` и `parent`.

Тестирование

В рамках тестирования разработанной программы для варианта 4а (добавлена операция замены одного символа на два) был создан набор unit-тестов на основе фреймворка Google Test. Тесты проверяют корректность функции `minEditCost`, которая реализует алгоритм динамического программирования с учётом четырёх операций: удаление, вставка, замена ($1 \rightarrow 1$) и замена одного символа на два ($1 \rightarrow 2$). Ниже приведено описание каждого теста.

1. `StandardLevenshtein` проверяет поведение алгоритма, когда операция замены $1 \rightarrow 2$ невыгодна (её стоимость установлена высокой, равной 100). В этом случае алгоритм должен выдать классическое расстояние Левенштейна для трёх известных пар строк: "kitten" $\rightarrow$ "sitting" (расстояние 3), "sunday" $\rightarrow$ "saturday" (расстояние 3), "pedestal" $\rightarrow$ "stien" (расстояние 7). Тест подтверждает, что при невыгодной дополнительной операции результаты совпадают с эталонными.

2. `Replace1to2Cheaper` проверяет ситуацию, когда операция замены $1 \rightarrow 2$ выгоднее стандартной комбинации (замена + вставка). Для строк "x" $\rightarrow$ "yz" и "a" $\rightarrow$ "bc" при стоимости замены $1 \rightarrow 2 = 1$, а стоимости всех остальных операций = 1 ожидаемая минимальная стоимость равна 1 (используется одна операция $1 \rightarrow 2$). Тест убеждается, что алгоритм действительно выбирает эту операцию и даёт правильную стоимость.

3. `Replace1to2NotUsed` проверяет обратную ситуацию – когда операция $1 \rightarrow 2$ дороже (стоимость 3), чем стандартные (стоимости 1). Для тех же пар строк "x" $\rightarrow$ "yz" и "a" $\rightarrow$ "bc" ожидается стоимость 2 (например, замена + вставка). Тест гарантирует, что алгоритм не использует дорогую операцию и возвращает корректную минимальную стоимость.

4. `EmptyStrings` проверяет работу алгоритма на граничных случаях с пустыми строками. Проверяются три сценария: обе строки пусты $\rightarrow$ стоимость 0; первая строка не пуста, вторая пуста $\rightarrow$ стоимость равна $\text{len}(A) * \text{cost_delete}$

(здесь 3); первая строка пуста, вторая не пуста $\rightarrow$ стоимость равна $\text{len}(B) * \text{cost_insert}$ (здесь 3). Тест подтверждает правильную обработку базовых случаев динамического программирования.

5. Identity проверяет поведение для одинаковых строк. Для строк "abc" и "abc" при любых разумных стоимостях (в тесте замены 5, вставки/удаления 5, замена $1 \rightarrow 2 = 100$) ожидается стоимость 0. Также проверяется пустой случай. Тест убеждается, что алгоритм не добавляет лишних операций при совпадении символов.

6. OnlyInsertDelete проверяет ситуации, когда выгодны только вставки и удаления, а замена дорогая (стоимость замены $1 \rightarrow 1 = 10$, вставки и удаления = 1, замена $1 \rightarrow 2$ дорогая = 100). Для преобразования "ab" $\rightarrow$ "abc" ожидается одна вставка (стоимость 1), для "abc" $\rightarrow$ "ab" – одно удаление (стоимость 1). Тест проверяет, что алгоритм не использует дорогие замены и правильно выбирает вставки/удаления.

7. Replace1to2WithLongerStrings проверяет применение операции $1 \rightarrow 2$ в более сложном примере: преобразование "ab" $\rightarrow$ "cde". При стоимости всех операций = 1 оптимальный план: заменить 'a' на "cd" (стоимость 1) и заменить 'b' на "e" (стоимость 1), итого 2. Альтернативный стандартный путь (замена+вставка) дал бы 3. Тест проверяет, что алгоритм находит стоимость 2 и, следовательно, использует хотя бы одну операцию $1 \rightarrow 2$.

8. UseMultipleReplace1to2 проверяет возможность применения нескольких операций замены $1 \rightarrow 2$ подряд. Строка "xy" преобразуется в "abcd". При стоимости всех операций = 1 можно заменить 'x' на "ab" и 'y' на "cd", получив стоимость 2. Стандартными операциями потребовалось бы 4. Тест подтверждает, что алгоритм способен использовать две операции $1 \rightarrow 2$ и даёт правильную стоимость.

9. EdgeCaseJLessThan2 проверяет граничные случаи, когда индекс j во внутреннем цикле меньше 2, и операция замены $1 \rightarrow 2$ недоступна (чтобы избежать выхода за границы массива $B[j-2]$). Проверяются три случая: замена

одного символа на один ("a" → "b"); удаление ("a" → ""); вставка (" " → "b"). Во всех случаях ожидается стоимость 1, и алгоритм не должен обращаться к j-2. Тест проверяет, что программа корректно обрабатывает такие ситуации без ошибок.

Все 9 тестов успешно пройдены. Это подтверждает, что реализованный алгоритм динамического программирования с дополнительной операцией замены 1→2 работает правильно во всём диапазоне входных данных, включая граничные условия и случаи, требующие принятия решений о выгодности использования новой операции.

Вывод

В ходе выполнения лабораторной работы были успешно реализованы четыре алгоритма на основе метода динамического программирования для расчёта редакционного расстояния между строками. Решены следующие задачи: разработана программа для вычисления взвешенного редакционного расстояния (Вагнер–Фишер) с произвольными стоимостями операций вставки, удаления и замены. Алгоритм корректно обрабатывает строки длиной до 2550 символов, имеет временную сложность $O(n*m)$ и использует память $O(n*m)$. Реализовано восстановление редакционного предписания – последовательности операций, превращающей строку А в строку В с минимальной стоимостью. Создана программа для классического расстояния Левенштейна (единичные стоимости). Реализован расширенный вариант 4а с добавлением операции замены одного символа на два. В программу включён подробный вывод промежуточных таблиц (dp и parent), легенды и последовательности операций. Это позволяет наглядно отследить ход вычислений и убедиться в выборе оптимального пути. Асимптотическая сложность осталась $O(n*m)$ за счёт константной проверки четвертой операции. Проведено автоматизированное тестирование с использованием Google Test. Набор из 9 тестов проверяет: корректность работы на классических примерах (Левенштейн); выгодность и невыгодность операции $1 \rightarrow 2$; обработку пустых строк и одинаковых строк; граничные случаи ($j < 2$); использование нескольких операций $1 \rightarrow 2$ подряд. Все тесты успешно пройдены, что подтверждает надёжность и правильность реализаций.